Kosuke Kikuchi

Nishinomiya, Hyogo, Japan Platform / Backend Engineer | Architecture, Data Integrity, AI-assisted Delivery GitHub: https://github.com/kousukekikuchi1984

Last updated: September 5, 2026

Summary

Platform and backend engineer specializing in **mission-critical data integrity**. I design systems where incorrect data must not become “confirmed truth,” such as accounting outputs and point-balance ledgers. I prioritize **correctness, reproducibility, and explainability** across retries, partial failure, asynchronous boundaries, and long-term operations.

I currently work at **NTT-ME Corporation** on the **AI Property Resilience Business**, transferred from Tractable in the UK. In my first three months, I delivered a time-sensitive PoC UI change, defined root-cause remediation for an OpenSearch capacity incident, restored a serverless DWH, eliminated recurring Terraform drift, clarified underspecified requirements, and built an LLM-assisted review environment. The DWH implementation, infrastructure, and data validation took five working days; the end-to-end effort, including requirements and architecture, took about one month. I validated counts, missing records, duplicates, and processing time. I now work as an IC supporting technical problem solving, project management, and ML delivery while practicing spec-driven development and reviewing LLM-assisted code for architecture, security, execution-model, and operational risks.

Core Strengths

- **Data integrity design:** idempotency, state transitions, failure modes, re-execution strategy, auditability
- **System takeover:** architecture discovery, operational risk mapping, requirements/SLA discovery, external-service inventory
- **Production troubleshooting:** root-cause analysis (DB deadlocks, memory leaks), prevention by structural fixes
- **Backend architecture:** long-term maintainability, clear boundaries of responsibility, operationally sound workflows
- **Cross-functional delivery:** align stakeholders by clarifying constraints, risks, and alternatives (servant-style leadership)
- **Technical requirements:** turn ambiguous requests into scoped, executable work with explicit risks and completion criteria
- **Spec-driven LLM engineering:** define scope, invariants, failure behavior, and acceptance criteria before using Cursor / LLM, then review implementation against the specification

Technical Skills

- **Backend:** Python (primary), Rust / Go / Java (limited)
- **Frontend:** TypeScript (production), Flutter (limited)
- **Databases/Data:** MySQL (primary), DynamoDB, OpenSearch, DuckDB, AWS Athena
- **Cloud/Infra:** AWS (ECS, EKS, Lambda, S3, CloudFront, CloudWatch, EventBridge, Step Functions), Terraform
- **Data tools:** Digdag (code-level analysis), Embulk (code-level analysis)
- **Other:** HULFT (file transfer integration), Linux (conceptual familiarity)
- **AI-assisted development:** Cursor / LLM (spec-driven development, codebase discovery, scoped implementation, generated-code review)

Experience

NTT-ME Corporation — Software Engineer

Jun 2026 – Present

Department: Social Infrastructure Design Department, AI Property Resilience Business

Scope: Software engineering for the AI Property Resilience Business, transferred from Tractable in the UK. I am taking over the inherited system while acting as a technical requirements and quality gate across architecture, security, data, and operations.

Selected accomplishments in the first three months

- **Delivered a PoC UI change in approximately one week:** started in the middle of my

second week, used Cursor for codebase navigation and implementation support, and completed the business-validation change by the middle of my third week.

- **Restored a serverless DWH in about one month:** used DynamoDB Export,

S3, EventBridge, Step Functions, and Athena to restore the data foundation without always-on servers. Implementation, infrastructure, and data validation took five working days; requirements and architecture accounted for the remaining end-to-end effort. Reconciled counts, missing records, duplicates, and processing time. Once BI adoption is complete, it is expected to remove one to two hours of weekly CSV aggregation and support SLA reporting.

- **Shifted an OpenSearch capacity incident from timeout tuning to root-cause remediation:**

identified under-provisioned capacity, evaluated cost and rollout options, defined the Terraform / tfstate reconciliation approach, and presented the risks to the Engineering Director.

- **Turned an underspecified AWS security request into executable work:** assumed technical

ownership, clarified requirements, scope, and the response approach, and moved the work into execution.

- **Reset requirements for a client PoC before implementation:** prevented premature

implementation by aligning stakeholders on the actual requirements, and decomposed the outcome into actionable tasks with completion criteria.

- **Built an LLM-assisted review environment:** created a Cursor-based CI/CD review workflow.

In one measured case, review time fell from approximately 60 minutes to five minutes while I still verified the change purpose, execution path, permission and data impact, and failure modes.

- **Restored reliable Terraform plan and apply:** resolved root causes across S3, Lambda,

Cognito, and EC2 that caused drift, repeated diffs, or failed applies. Verified clean plans when no change exists and successful apply execution.

- **Assessed development readiness across five initiatives:** found that four were not ready

for engineering commitment because of contract, problem-definition, scope, requirement, ownership, or funding gaps, and made the blockers explicit before delivery planning.

Current responsibilities

- Work as an IC alongside the PdM and TL/PM, solving technical problems and supporting

project-management and ML delivery needs.

- Understand the existing architecture, codebase, data flows, and operational workflows.
- Identify technical risks, knowledge concentration, and improvement opportunities required for stable post-transfer operations.
- Organize business, operational, and technical assumptions into practical engineering context for future development and maintenance.
- Built the serverless DWH, data validation, and operating design, and now drive BI selection,

connection, and adoption through actual use.

- Design DynamoDB -> OpenSearch DLQ replay rules using version-based optimistic locking and a notification-first rollout.
- Assess AWS networking impact for a 2VPC -> 3VPC transition across ECS, S3, DynamoDB, Lambda, and OpenSearch.
- Convert ambiguous business or project requests into executable technical work and review

architecture, security, data, operational risk, and generated-code behavior.

- Practice spec-driven development on real changes, with team sharing after repeated success

and automated review planned for stable, high-signal checks.

Current design themes

- **Post-transfer ownership:** classify unknowns as confirmed / unconfirmed / inferred, and build enough operational context to release, investigate incidents, and reason about rollback.
- **DWH/BI adoption:** completed the serverless data foundation and validation without

recreating the previous always-on design. BI selection, connection, and operational adoption remain before the expected manual-work reduction and SLA-reporting outcome are realized.

- **DLQ conflict handling:** prevent stale replay events from overwriting newer state by using version checks, update-path conflict rules, and manual review for overlapping changes.
- **AWS networking:** evaluate routing, VPC endpoints, security groups, DNS, IAM conditions, and service-level impact before changing VPC topology.
- **Low-utilization cost architecture:** evaluate consolidating low-frequency API functions

where Provisioned Concurrency creates fixed idle cost, and compare Lambda with on-demand container execution for sparse ML workloads. Savings remain estimates pending workload and billing validation.

- **Spec-driven AI-assisted development:** define specification boundaries before generation,

compare implementation against intended behavior, and evaluate leverage through lead time, rework, review cost, defects, and production outcomes rather than token volume. For the DWH, two to three independent generation cycles converged on functionally equivalent behavior that passed tests and runtime validation.

YUZURIHA — Engineer (Backend / Platform)

Jan 2021 – May 2026

Scope: Technical direction for a ~10-engineer product team; design/code reviews as a quality gate; sub-project leadership (3–4 people); first-round engineering interviews.

Selected accomplishments

- **Prevented accounting CSV misstatement before launch** by identifying a latent edge case and correcting the design to match accounting rules (avoided costly post-release reconciliation).
- **Built safe large-scale ingestion for credit-card data (300K records, 1-hour SLA)** by redefining inconsistent external rules, deriving a safe parallelization unit (user-level key), separating validation from apply, and enforcing idempotent re-execution; achieved ~6 minutes ingestion in practice.
- **Led high-risk cutover of file-transfer integration (HULFT / OS / crypto changes)** across 4 stakeholders; re-evaluated trade-offs when new constraints emerged and executed a one-shot cutover with rollback/monitoring plans—completed without critical incidents.
- **Stopped unsafe “performance optimization”** by proving the bottleneck was MySQL commit fsync (not app CPU), proposing alternatives (commit reduction, responsibility split), and preventing a design fork that risked data integrity.
- **Resolved production incidents structurally:** fixed a post-release **memory leak** caused by persistent caching of DB Engine objects; eliminated recurring **DB deadlocks** by standardizing lock acquisition order and codifying the rule.

Gunosy — Data Reliability Engineer

May 2019 – Dec 2020 Ensured correctness and explainability across large-scale data workflows spanning ingestion, aggregation, and business operations.

- Designed revenue data finalization workflows with strict “freeze” semantics and resolved non-obvious Digdag behavior via code-level analysis.

• Built workflow templates to democratize the platform and reduced operational noise by designing convergence-by-structure.

Reference: https://tech.gunosy.io/entry/escape-from-pitfall

PERSOL CAREER — Systems Engineer

Oct 2017 – Apr 2019 Selected: on-prem Hadoop for ML systems; docs for ML project execution; engine maintenance & review.

IP ONWEB JAPAN — Data Mining Engineer

Feb 2016 – Sep 2017 Analytics/ops for advertising delivery and revenue data; anomaly detection from distribution shifts; root-cause analysis of “looks normal but wrong” data states.

Spotlight — Engineer

Apr 2014 – Jan 2016 O2O product development; improved report generation performance and operational efficiency.

Technical Writing & Contributions

• “■■■■■■■■■■■■■■■■■” (Zenn):

https://zenn.dev/kou2kkkt/articles/d52dca7e2a1872

• Examines AI-era Software Factories by separating execution and verification loops

from individual and organizational learning loops.

• “■■■■■■■■■■■■■■■” (Zenn):

https://zenn.dev/kou2kkkt/articles/fc92cd9d6599fb

• Uses Xenakis's work across music, architecture, and mathematics to examine the distinct

abilities to design generative rules and evaluate generated outcomes.

• HTTP/3 (QUIC) partial Japanese translation: https://http3-explained.haxx.se/ja

• Cache locality and performance reasoning (Qiita): https://qiita.com/kikuchi-yzrh/items/56b2368e4473e19ec1d9

• SQLAlchemy IdentityMap deep-dive (Qiita): https://qiita.com/kikuchi-yzrh/items/9680fd85f0dcb474ef18

• MySQL full-text search edge case note (Qiita): https://qiita.com/kikuchi-yzrh/items/baaab16cc1258e81e476

• Digdag retry duplication pitfall (Gunosy Tech Blog): https://tech.gunosy.io/entry/escape-from-pitfall